

CONFIDENCIAL · USO INTERNO

Relatório de Auditoria de Segurança pdv-web

Revisão adversarial de 5 categorias de falha comuns — isolamento de tenant, autorização, IDOR, segredos e XSS — adaptadas à stack real do projeto.

Data	2026-09-17
Escopo	apps/api (NestJS + Prisma/PostgreSQL), apps/web (Next.js), packages/shared (Zod), infraestrutura (docker-compose, CI, scripts de deploy)
Achados	4 (1 crítico, 1 alto)
Metodologia	stack detectada: NestJS + Prisma/PostgreSQL (RLS) no backend, Next.js/React no frontend, Zod para validação compartilhada, Docker Compose + GitHub Actions para deploy/CI. Ver tabela de mapeamento na próxima página.

Nota metodológica

Banco sem tranca (isolamento de tenant)	Row-Level Security nativa do PostgreSQL (FORCE ROW LEVEL SECURITY) por tabela, ativada via `app.tenant_id` setado por uma extensão do Prisma Client (AsyncLocalStorage), somada a filtro explícito por tenantId em todo Repository.
Permissão definida no navegador	Guards do NestJS (AuthGuard, `RolesGuard`, `PlatformAuthGuard`) registrados globalmente ou por controller/rota; comparado com os `role === 'ADMIN'` do frontend (React) que só escondem UI.
IDOR	Todo handler que recebe `:id` foi percorrido nos 8 controllers (`tenant`, `product`, `cash-session`, `sale`, `operator`, `storage`, `dashboard`, `reports`, `platform-*`), cruzando com o Repository correspondente.
Chaves expostas	`.env.example` (raiz, apps/api, apps/web), `docker-compose.yml`/`docker-compose.dev.yml`, `.github/workflows/ci.yml`, `scripts/deploy-check.sh`, histórico do git (`git grep`/`git log` por padrões de chave e por `.env` commitado).
Inputs sem tratamento (XSS)	Frontend Next.js/React: busca por `dangerouslySetInnerHTML`, `eval`, libs de markdown/HTML, e todo `href`/`src` dinâmico. Backend: template de e-mail (MailService) — texto puro, sem HTML.

Resumo executivo



Pontos fortes

Row-Level Security forçada em todas as tabelas de domínio, com defesa em profundidade no Repository

apps/api/prisma/migrations/20260912180000_row_level_security/migration.sql (ENABLE + FORCE ROW LEVEL SECURITY em Operator, Product, CashSession, Sale, SaleItem, PinToken); todo método de apps/api/src/modules/*/*.repository.ts que busca por id também filtra por tenantId explicitamente (ex.: operator.repository.ts:47, product.repository.ts:24, cash-session.repository.ts:51) — mesmo se a extensão RLS falhasse, o filtro explícito ainda barra o cross-tenant.

SaleItem sem tenantId próprio, isolado transitivamente por EXISTS na Sale (que já tem RLS)

migration.sql: `CREATE POLICY tenant\_isolation ON "SaleItem" USING (EXISTS (SELECT 1 FROM "Sale" s WHERE s.id = "SaleItem"."saleId"))` — o subselect em Sale herda a política de Sale, então só enxerga itens de vendas do tenant atual.

Guards de autorização aplicados de forma consistente e cruzados corretamente com o frontend

apps/api/src/app.module.ts:64-66 registra AuthGuard e RolesGuard globalmente; todo endpoint de escrita sensível (product.controller.ts:54/65/77, operator.controller.ts:38, storage.controller.ts:29, dashboard.controller.ts:13, reports.controller.ts:14, tenant.controller.ts:42) tem `@Roles('ADMIN')` no backend, não confiando só no isAdmin do React.

Upload de imagem via URL assinada com validação real do objeto (não confia no Content-Type declarado)

apps/api/src/modules/storage/storage.service.ts:31-42 confere prefixo `tenants/<tenantId>/`, HEAD do objeto, magic bytes (JPEG/PNG/WebP) e tamanho máximo — apaga e rejeita se algo não bater.

CORS com regex de subdomínio corretamente escapado

apps/api/src/common/cors.ts escapa todos os metacaracteres antes de trocar `\*` por `[^.]` — não há brecha de regex-injection na lista de origens permitidas.

Sessão de superadmin isolada da sessão de loja (segredo JWT e cookie próprios, guard próprio, não-global)

apps/api/src/modules/platform/platform.module.ts:15-27 usa PLATFORM_SESSION_SECRET dedicado num JwtModule não-global; PlatformAuthGuard nunca chama getTenantId().

Sem HTML em e-mail transacional — nenhuma superfície de injeção de template

apps/api/src/modules/mail/mail.service.ts envia só `text`, nunca `html`; nenhum ponto do código monta HTML para e-mail.

Segredos e .env corretamente fora do controle de versão, sem vazamento no histórico

.gitignore:5,7 ignora apps/api/.env e apps/web/.env.local; `git log --all` para esses arquivos não retorna nenhum commit; `git grep` por padrões de chave (AWS AKIA, sk-, AIza) não encontrou nada no código rastreado.

Respostas de autenticação genéricas contra enumeração de conta (timing e mensagem)

platform-auth.service.ts:52-56 roda argon2.verify contra um dummyHash mesmo quando o e-mail não existe (mesmo padrão em auth.service.ts) — mesma mensagem/tempo de resposta para senha errada e e-mail inexistente, em ambos os logins (operador e superadmin).

Achados detalhados

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
CRÍTICA	scripts/deploy-check.sh:7	PLATFORM_SESSION_SECRET fica fora da validação do <code>deploy-check.sh</code> — pode ir para produção com o valor placeholder do <code>.env.example</code> CHAVES EXPOSTAS <pre>required="APP_DOMAIN API_DOMAIN MEDIA_DOMAIN ACME_EMAIL POSTGRES_PASSWORD APP_DB_PASSWORD STORAGE_SECRET_KEY SESSION_SECRET"</pre> O script que barra `docker compose up` com placeholder no <code>.env</code> (comentário do próprio <code>docker-compose.yml</code>: "rode <code>./scripts/deploy-check.sh</code> antes de <code>docker compose up</code>") lista <code>SESSION_SECRET</code> mas esquece <code>PLATFORM_SESSION_SECRET</code> — o segredo que assina o JWT da sessão de Superadmin <pre>(apps/api/src/modules/platform/platform.module.ts:24, `config.getOrThrow('PLATFORM_SESSION_SECRET')`).</pre> Não existe nenhuma outra validação de startup que rejeite o valor `change-me-to-a-different-long-random-secret` do <code>apps/api/.env.example:39</code> — <code>getOrThrow</code> só verifica presença, não força. Se quem faz o deploy copiar o <code>.env.example</code> e esquecer essa linha (fácil, já que o script 'passa' mesmo assim), qualquer pessoa com o texto do <code>.env.example</code> público neste repositório pode forjar um cookie <code>pdv_platform_session</code> válido para QUALQUER PlatformAdmin (<code>{ sub: <id> }</code>), sem verificação de senha) — controle total do painel Superadmin: criar/suspender lojas, ver todas as lojas cadastradas. Condição de exploração: Só se explora se o operador do deploy não sobrescrever <code>PLATFORM_SESSION_SECRET</code> no <code>.env</code> de produção — mas nada no processo de deploy documentado (<code>deploy-check.sh</code>) obriga isso, ao contrário do que já é feito para <code>SESSION_SECRET</code>.
ALTA	docker-compose.yml:49, 68, 71, 88	Defaults de senha fracos e públicos para Postgres e MinIO embutidos direto no <code>docker-compose.yml</code> de produção CHAVES EXPOSTAS <pre>DATABASE_URL: postgresql://\${APP_DB_USER:- pdv}:\${APP_DB_PASSWORD:-pdv}@postgres:5432/... POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:-postgres} APP_DB_PASSWORD: \${APP_DB_PASSWORD:-pdv} MINIO_ROOT_PASSWORD: \${STORAGE_SECRET_KEY:-pdvpdpdpv}</pre> Esses três defaults (<code>postgres</code>, <code>pdv</code>, <code>pdvpdpdpv</code>) SÃO cobertos pelo <code>deploy-check.sh</code> (achado positivo, ver seção de pontos fortes) — mas o <code>deploy-check.sh</code> só é chamado se o operador lembrar de rodá-lo manualmente antes do <code>docker compose up -d --build</code> (não há hook automático, nem o próprio <code>compose</code> recusa subir sem ele). Um <code>docker compose up</code> direto, sem passar pelo script, sobe com Postgres e MinIO usando essas senhas triviais, expostas neste repositório público — se a porta do Postgres/MinIO acabar acessível na rede (erro comum de firewall em VPS), é acesso direto ao banco todo (bypassa até a RLS, que só vale para o usuário da aplicação, não para <code>POSTGRES_PASSWORD</code> que é superusuário). Condição de exploração: Precisa de um deploy que pule <code>scripts/deploy-check.sh</code>. E que a porta 5432/9000 fique acessível externamente (não expostas por padrão neste <code>docker-compose.yml</code>, mas comum em setups customizados de VPS).
BAIXA	packages/shared/src/schemas/tenant.ts:16, 31	logoUrl/photoUrl aceitam esquema javascript: na validação (<code>z.string().url()</code> não restringe protocolo) INPUTS SEM TRATAMENTO (XSS) <pre>logoUrl: z.string().url().nullable(), logoUrl: z.string().url('URL de logo inválida.').nullable(),</pre> O validador <code>.url()</code> do Zod aceita qualquer URL sintaticamente válida, incluindo o esquema <code>javascript:</code> (ex.: <code>javascript:alert(1)</code> passa). O mesmo padrão se repete em <code>product.ts</code>, <code>operator.ts</code>, <code>auth.ts</code>, <code>report.ts</code>, <code>dashboard.ts</code> para <code>photoUrl</code>. Na prática, hoje isso NÃO é explorável: todo valor chega por upload real (<code>packages/PhotoUploadBox</code> -> <code>POST /uploads/confirm</code>, que devolve uma <code>publicUrl</code> do próprio bucket) e o único consumo no frontend é <code><img src=</code> (Avatar.tsx:22, TenantMark.tsx:8, ProductCard.tsx:22 etc.), <code>manifest.ts</code> (campo JSON <code>src</code> do Web App Manifest) e <code>layout.tsx</code> (metadada de <code><link rel=icon></code>) — nenhum desses é um sink que executa <code>javascript:</code>. O risco é teórico/defesa-em-profundidade: um PATCH direto na API (fora da UI) por um Administrador da própria loja poderia gravar <code>javascript:...</code> no

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
		campo, e se um consumidor futuro (ex.: um `<a a="" adicionado="" em="" essa="" for="" foto="" href='{logoUrl}>"ver' p="" real")`="" real.<="" restrição,="" se="" sem="" tamanho="" tornaria="" xss=""> Condição de exploração: Não é explorável nos sinks atuais (`<img src>`, `manifest`, `metadata` de ícone). Vira explorável apenas se um `<a adicionado="" adicional.<="" campos="" esses="" for="" href>"="" p="" redirect`="" sanitização="" sem="" usando="" window.open"="">
INFORMATIVA	apps/api/src/modules/platform/platform-tenant.service.ts:128-137	Checagem de e-mail de administrador único entre lojas (isAdminEmailInUse) tem race condition TOCTOU — já mapeada pelo time, sem fix aplicado BANCO SEM TRANCA / IDOR <pre>private async isAdminEmailInUse(email: string): Promise<boolean> { const tenants = await this.prisma.tenant.findMany({ select: { id: true } }) for (const tenant of tenants) { const found = await tenantStorage.run({ tenantId: tenant.id }, async () => { return this.prisma.operator.findFirst({ where: { tenantId: tenant.id, email, deletedAt: null } }) }) if (found) return true } return false }</pre> Diferente do achado já corrigido nesta mesma sessão para PlatformAdmin.email (que tem @unique no banco e agora vira 409 tratado), Operator.email só é único por tenant (`@@unique({tenantId, email})`) — não há constraint global no banco para pegar a corrida entre o check (findFirst) e a criação da loja/operador. Duas chamadas concorrentes de POST /platform/tenants com o mesmo adminEmail em lojas diferentes podem, em teoria, passar as duas pela checagem antes de qualquer uma criar o operador, resultando no mesmo e-mail administrando duas lojas — quebra de uma regra de negócio (não de isolamento de dado entre tenants, e não path de dado sensível cross-tenant), não uma falha de segurança direta. Severidade rebaixada a informativa: já era um item conhecido e aceito nesta sessão (surface de uso é só o Superadmin, ator único e de confiança), citado aqui só para constar na cobertura completa da auditoria. Condição de exploração: Exige que o próprio Superadmin (único ator com acesso a esta rota) dispare duas criações de loja simultâneas com o mesmo e-mail — não é um vetor de ataque externo.

Recomendações priorizadas

P1 Adicionar PLATFORM_SESSION_SECRET à validação obrigatória do deploy-check.sh

Uma linha em scripts/deploy-check.sh (achado F1) — mesma lista `required` que já cobre SESSION_SECRET. Sem custo de implementação, fecha o maior risco da auditoria.

P1 Adicionar validação de startup que rejeite qualquer segredo com valor de placeholder

Complementar ao P1 acima: um `if` no bootstrap (main.ts) que recusa subir se SESSION_SECRET/PLATFORM_SESSION_SECRET baterem em `change-me-to-\*` — defesa mesmo se alguém rodar docker compose sem passar pelo deploy-check.sh (achado F1/F2).

P2 Remover (ou documentar explicitamente como "só localhost") os defaults de senha do docker-compose.yml

POSTGRES_PASSWORD, APP_DB_PASSWORD, STORAGE_SECRET_KEY (achado F2) — hoje o comentário no topo do arquivo já avisa, mas o fallback funcional ainda existe; considerar falhar o container se a env vier vazia em vez de usar `postgres`/`pdv`/`pdvpdpdv`.

P3 Restringir o schema de logoUrl/photoUrl a protocolo http(s)

Trocar `z.string().url()` por um refine que rejeite qualquer esquema fora de http/https em packages/shared/src/schemas/*.ts (achado F3) — defesa em profundidade, sem sink explorável hoje.

P3 Avaliar lock/transação para isAdminEmailInUse em PlatformTenantService.create()

Baixo risco (ator único, superadmin) — considerar uma constraint auxiliar (ex.: tabela global de e-mails de admin) só se o produto crescer para múltiplos operadores de plataforma (achado F4).

Issues para o GitHub

Texto pronto para copiar e colar na criação de issues — um bloco por achado acionável (achados informativos, sem ação de código necessária, ficam de fora).

```
--- ISSUE 1 ---
## [Segurança] PLATFORM_SESSION_SECRET fica fora da validação do deploy-check.sh — pode ir para produção com o valor placeholder do .env.example

**Labels sugeridas:** security, critica

### Descrição do problema
O script que barra `docker compose up` com placeholder no .env (comentário do próprio docker-compose.yml: "rode ./scripts/deploy-check.sh antes de docker compose up") lista SESSION_SECRET mas esquece PLATFORM_SESSION_SECRET — o segredo que assina o JWT da sessão de Superadmin (apps/api/src/modules/platform/platform.module.ts:24, `config.getOrThrow('PLATFORM_SESSION_SECRET')`). Não existe nenhuma outra validação de startup que rejeite o valor `change-me-to-a-different-long-random-secret` do apps/api/.env.example:39 — getOrThrow só verifica presença, não força. Se quem faz o deploy copiar o .env.example e esquecer essa linha (fácil, já que o script 'passa' mesmo assim), qualquer pessoa com o texto do .env.example público neste repositório pode forjar um cookie pdv_platform_session válido para QUALQUER PlatformAdmin (`{ sub: <id> }`, sem verificação de senha) — controle total do painel Superadmin: criar/suspender lojas, ver todas as lojas cadastradas.

### Evidência
`scripts/deploy-check.sh:7`
...
required="APP_DOMAIN API_DOMAIN MEDIA_DOMAIN ACME_EMAIL POSTGRES_PASSWORD APP_DB_PASSWORD STORAGE_SECRET_KEY SESSION_SECRET"
...

### Impacto
Severidade: **CRÍTICA**. Só se explora se o operador do deploy não sobrescrever PLATFORM_SESSION_SECRET no .env de produção — mas nada no processo de deploy documentado (deploy-check.sh) obriga isso, ao contrário do que já é feito para SESSION_SECRET.

### Sugestão de correção
Uma linha em scripts/deploy-check.sh (achado F1) — mesma lista `required` que já cobre SESSION_SECRET. Sem custo de implementação, fecha o maior risco da auditoria.

### Critérios de aceite
- [ ] Causa raiz corrigida no arquivo/linha indicado
- [ ] Teste (unitário ou de integração) cobrindo o caso, quando aplicável
- [ ] Revisão confirma que a condição de exploração deixou de existir
--- FIM ISSUE 1 ---

--- ISSUE 2 ---
## [Segurança] Defaults de senha fracos e públicos para Postgres e MinIO embutidos direto no docker-compose.yml de produção

**Labels sugeridas:** security, alta

### Descrição do problema
Esses três defaults (`postgres`, `pdv`, `pdvpdpdpdv`) SÃO cobertos pelo deploy-check.sh (achado positivo, ver seção de pontos fortes) — mas o deploy-check.sh só é chamado se o operador lembrar de rodá-lo manualmente antes do `docker compose up -d --build` (não há hook automático, nem o próprio compose recusa subir sem ele). Um `docker compose up` direto, sem passar pelo script, sobe com Postgres e MinIO usando essas senhas triviais, expostas neste repositório público — se a porta do Postgres/MinIO acabar acessível na rede (erro comum de firewall em VPS), é acesso direto ao banco todo (bypassa até a RLS, que só vale para o usuário da aplicação, não para POSTGRES_PASSWORD que é superusuário).

### Evidência
`docker-compose.yml:49, 68, 71, 88`
...
DATABASE_URL: postgresql://${APP_DB_USER:-pdv}:${APP_DB_PASSWORD:-pdv}@postgres:5432/...
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
APP_DB_PASSWORD: ${APP_DB_PASSWORD:-pdv}
MINIO_ROOT_PASSWORD: ${STORAGE_SECRET_KEY:-pdvpdpdpdv}
...

### Impacto
Severidade: **ALTA**. Precisa de um deploy que pule scripts/deploy-check.sh E que a porta 5432/9000 fique acessível externamente (não expostas por padrão neste docker-compose.yml, mas comum em setups customizados de VPS).

### Sugestão de correção
Complementar ao P1 acima: um `if` no bootstrap (main.ts) que recusa subir se SESSION_SECRET/PLATFORM_SESSION_SECRET baterem em `change-me-to-*` — defesa mesmo se alguém rodar docker compose sem passar pelo deploy-check.sh (achado F1/F2).

### Critérios de aceite
- [ ] Causa raiz corrigida no arquivo/linha indicado
- [ ] Teste (unitário ou de integração) cobrindo o caso, quando aplicável
- [ ] Revisão confirma que a condição de exploração deixou de existir
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
## [Segurança] logoUrl/photoUrl aceitam esquema javascript: na validação (z.string().url()) não restringe protocolo)

**Labels sugeridas:** security, baixa

### Descrição do problema
O validador `.url()` do Zod aceita qualquer URL sintaticamente válida, incluindo o esquema `javascript:` (ex.: `javascript:alert(1)` passa). O mesmo padrão se repete em product.ts, operator.ts, auth.ts, report.ts, dashboard.ts para `photoUrl`. Na prática, hoje isso NÃO é explorável: todo valor chega por upload real (packages/PhotoUploadBox -> POST /uploads/confirm, que devolve uma publicUrl do próprio bucket) e o único consumo no frontend é `` (Avatar.tsx:22, TenantMark.tsx:8, ProductCard.tsx:22 etc.), `manifest.ts` (campo JSON `src` do Web App Manifest) e `layout.tsx` (metadata de `

```

```
logoUrl: z.string().url('URL de logo inválida.').nullable(),
...

### Impacto
Severidade: **BAIXA**. Não é explorável nos sinks atuais (`<img src>`, manifest, metadata de ícone). Vira explorável apenas se um `9 / 9
```